

**LOYOLA ACADEMY
DEGREE & PG COLLEGE
OLD ALWAL, SECUNDERABAD - 500 010,
TELANGANA, INDIA**

**An Autonomous Institution Affiliated to Osmania
University**

**Re-accredited with 'A' Grade (III Cycle) by NAAC A "College with Potential for
Excellence" by UGC**



Practical Record

CERTIFICATE

This is to certify that this is a Bonafide record work done in **DATABASE
SECURITY** practical during **III** year **V** semester of the academic year 2026-2027

Name:

UID No:

Class:

Signature of Internal

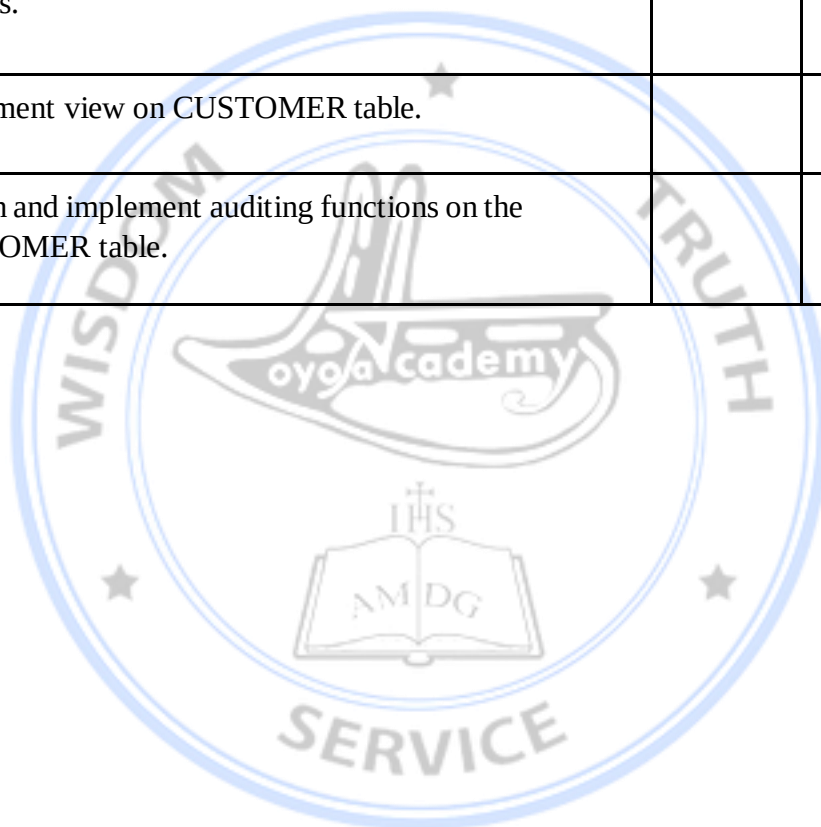
Signature of HoD

Signature of External

Signature of Principal

INDEX

S.No	Experiment Name	Date	Signature
1.	Create database users with the parameters specified.		
2.	Create a CUSTOMER table as a sample database object to implement security.		
3.	Create profiles and manage all database users using the profiles.		
4.	Implement view on CUSTOMER table.		
5.	Design and implement auditing functions on the CUSTOMER table.		



1. Create the following database users with the parameters specified in the table.

User	Password	Other Parameters
DBSEC_ADMIN	tec5363admin	Tablespace: USERS Temporary tablespace: TEMP
DBSEC_CLERK	tec5363clerk	Same as above
DBSEC_DEV	tec5363dev	Same as above
DBSEC	tec5363#1	Same as above
VPD_CLERK1	jessie\$22	Same as above
VPD_CLERK2	lassie\$46	Same as above

SQL Run SQL Command Line

```
SQL> CREATE USER DBSEC_ADMIN  
2 IDENTIFIED BY tec5363admin  
3 DEFAULT TABLESPACE USERS  
4 TEMPORARY TABLESPACE TEMP;
```

OUTPUT:

```
User created.
```

```
SQL> CREATE USER DBSEC_CLERK  
2 IDENTIFIED BY tec5363clerk  
3 DEFAULT TABLESPACE USERS  
4 TEMPORARY TABLESPACE TEMP;
```

OUTPUT:

```
User created.
```

```
SQL> CREATE USER DBSEC_DEV
```

2 IDENTIFIED BY tec5363dev
3 DEFAULT TABLESPACE USERS
4 TEMPORARY TABLESPACE TEMP;

OUTPUT:

```
User created.
```

SQL> CREATE USER DBSEC
2 IDENTIFIED BY tec536#1
3 DEFAULT TABLESPACE USERS
4 TEMPORARY TABLESPACE TEMP;

OUTPUT:

```
User created.
```

SQL> CREATE USER VPD_CLERK1
2 IDENTIFIED BY jessie\$22
3 DEFAULT TABLESPACE USERS
4 TEMPORARY TABLESPACE TEMP;

OUTPUT:

```
User created.
```

SQL> CREATE USER VPD_CLERK2
2 IDENTIFIED BY lassie\$46
3 DEFAULT TABLESPACE USERS
4 TEMPORARY TABLESPACE TEMP;

OUTPUT:

```
User created.
```

2. Create a CUSTOMER table as a sample database object to implement security. The DBSEC user is the owner of the CUSTOMER table, which has the following columns.

Column Name	Data Type
SALES_REP_ID	NUMBER(4)
CUSTOMER_ID	NUMBER(8) NOT NULL
CUSTOMER_SSN	VARCHAR2(9)
FIRST_NAME	VARCHAR2(20)
LAST_NAME	VARCHAR2(20)
ADDR_LINE	VARCHAR2(60)
CITY	VARCHAR2(30)
STATE	VARCHAR2(30)
ZIP_CODE	VARCHAR2(9)
PHONE	VARCHAR2(15)
EMAIL	VARCHAR2(80)
CC_NUMBER	VARCHAR2(20)
CREDIT_LIMIT	NUMBER
GENDER	CHAR(1)
STATUS	CHAR(1)
COMMENTS	VARCHAR2(1024)
CTL_UPD_DTTM	DATE
CTL_UPD_USER	VARCHAR2(30)
CTL_REC_STAT	CHAR(1)

Populate data using the file provided.

You may either put all "insert" statements within this file, or provide a link to another file. Refer to the dbsec_ch4_setup.sql file in your downloaded file if you like to see how it was done. If you use the link, please make sure your link works on your computer.

Create password complexity policy function (named "complexity_function"). The password policy will enforce the following complexity so that the password:

- Is at least six (6) characters long

- Differs from the username

- Has at least one alpha, one numeric, and one punctuation mark character
- Is not simple or obvious, such as welcome, account, oracle, database, or user.

- Differs from the previous password by at least 3 characters

SQL> Run SQL Command Line

SQL> create table customer (

sales_rep_id number(4),

customer_id number (8) not null,

customer_ssn varchar2 (9),

first_name varchar2 (20),

last_name varchar2(20),

addr_line varchar2(60),

city varchar2(30),

state varchar2(30),

zip_code varchar2(9),

phone varchar2(15),

email varchar2(80),

cc_number varchar2(20),

credit_limit number,

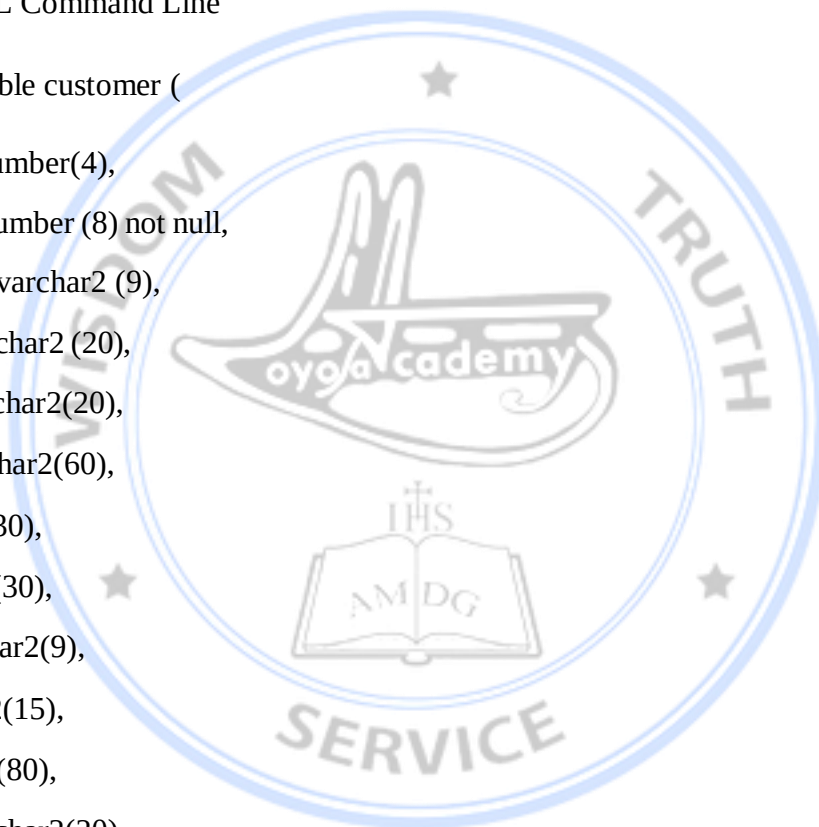
gender char(1),

status char(1),

comments varchar2(1024),

ctl_upd_dttm date,

ctl_upd_user varchar2(30),



```
ctl_rec_stat char(1));
```

OUTPUT:

```
Table created.
```

```
SQL> insert into customer values (1001,12345678,'999999999','john','doe','123 main st',  
'New York','NY','10001','123-456-7890','john.doe@example.com','4444',5000,'m','a',  
'first-time customer', to_date('2025-03-11','yyyy-mm-dd'),'admin','a');
```

OUTPUT:

```
1 row created.
```

```
SQL> insert into customer values (1002,11345678,'999999997','james','scott','123 main st',  
'New york','NY','10001','123-456-7890','james@example.com','4444',5000,'m','a','first-  
time customer', to_date('2025-03-11','yyyy-mm-dd'),'admin','a');
```

OUTPUT:

```
1 row created.
```

```
SQL> insert into customer values (1003,23456781,'888888888','jane','smith','1234 main st',  
'New york','NY','10002','123-456-7891','jane.smith@example.com','5555',6000,'m','a',  
'preferred customer', to_date('2025-03-11','yyyy-mm-dd'),'admin','a');
```

OUTPUT:

```
1 row created.
```

```
SQL> insert into customer values (1002,34567891,'777777777','alice','johnson','789 Oak  
st','Chicago','IL','10003','456-789-012','alice.johnson@example.com','3333',10000,  
'f','i','inactive customer', to_date('2025-03-11','yyyy-mm-dd'),'admin','T');
```

OUTPUT:

```
1 row created.
```

STEP 1 :CREATING Password_complexity FUNCTION

```

SQL> connect /as sysdba
Connected.
SQL> CREATE OR REPLACE FUNCTION password_complexity (
2     username      VARCHAR2,
3     password      VARCHAR2,
4     old_password  VARCHAR2
5 ) RETURN BOOLEAN IS
6 BEGIN
7     IF LENGTH(password) < 6 THEN
8         RAISE_APPLICATION_ERROR(-20001, 'Password must be at least 6 characters');
9     END IF;
10
11     IF LOWER(password) = LOWER(username) THEN
12         RAISE_APPLICATION_ERROR(-20002, 'Password must not be same as username');
13     END IF;
14
15     IF NOT REGEXP_LIKE(password, '[0-9]') THEN
16         RAISE_APPLICATION_ERROR(-20003, 'Password must contain at least one number');
17     END IF;
18
19     IF NOT REGEXP_LIKE(password, '[A-Za-z]') THEN
20         RAISE_APPLICATION_ERROR(-20004, 'Password must contain at least one letter');
21     END IF;
22     RETURN TRUE;
23 END;
24 /

Function created.

SQL> GRANT EXECUTE ON password_complexity TO PUBLIC;

Grant succeeded.

```

GRANT EXECUTE ON password_complexity TO PUBLIC;

But still, password functions **must be inside SYS schema.**

STEP 2 — Verify Function Exists

```

SELECT object_name, status
FROM dba_objects
WHERE object_name = 'PASSWORD_COMPLEXITY';

```

It should show:

```

Password_complexity      VALID

```

If you see 0 rows → the function is not in SYS.

```

SQL> SELECT object_name, status
2     FROM dba_objects
3     WHERE object_name = 'PASSWORD_COMPLEXITY';

OBJECT_NAME
-----
STATUS
-----
PASSWORD_COMPLEXITY
VALID

PASSWORD_COMPLEXITY
VALID

```

STEP 3 — Create Profile Using Your Function

```
CREATE PROFILE secure_profile  
LIMIT  
    PASSWORD_VERIFY_FUNCTION password_complexity;
```

```
SQL> CREATE PROFILE secure_profile1  
2  LIMIT  
3      PASSWORD_VERIFY_FUNCTION password_complexity;  
  
Profile created.
```

Assign Profile to User

```
ALTER USER your_username PROFILE secure_profile1;
```



3. Create profiles and manage all database users using the profiles including enforcing the password complexity.

You will create the following profiles:

Profile	Resources	Password
DBSEC_ADMIN_PROF	SESSIONS_PER_USER=5 CONNECT_TIME=8 hours IDLE_TIME=1 hour	PASSWORD_LIFE_TIME= 1 month PASSWORD_GRACE_TIME=7 days PASSWORD_VERIFY_FUNCTION=complexity_function
DBSEC_DEV_PROF	CONNECT_TIME=12 hours IDLE_TIME=2 hours CPU_PER_CALL=1 minute	PASSWORD_LIFE_TIME= 1 month PASSWORD_GRACE_TIME=14 days PASSWORD_VERIFY_FUNCTION=complexity_function
DBSEC_CLERK_PROF	SESSIONS_PER_USER=1 CPU_PER_CALL=5 seconds CONNECT_TIME=8 hours IDLE_TIME=30 minutes	FAILED_LOGIN_ATTEMPTS=3 PASSWORD_LIFE_TIME= 1 month PASSWORD_LOCK_TIME=3 days PASSWORD_GRACE_TIME=14 days
	LOGICAL_READS_PER_CALL=10 KB	PASSWORD_VERIFY_FUNCTION=complexity_function

You will create the following roles:

Role Name	Privileges
DBSEC_ADMIN_ROLE	SELECT and ALTER on all DBSEC tables
DBSEC_CLERK_ROLE	SELECT, INSERT, and UPDATE on all DBSEC tables
DBSEC_SUPERVISOR_ROLE	SELECT, INSERT, UPDATE and DELETE on all DBSEC tables
DBSEC_QUERY_ROLE	SELECT only on CUSTOMER table owned by DBSEC

You will assign roles and profiles to database users as follows:

User Name	Role	Profile
DBSEC_ADMIN	DBSEC_ADMIN_ROLE	DBSEC_ADMIN_PROF
DBSEC_CLERK, VPD_CLERK1, VPD_CLERK2	DBSEC_CLERK_ROLE	DBSEC_CLERK_PROF
DBSEC_DEV	DBSEC_ADMIN_ROLE plus DBSEC_SUPERVISOR_ROLE	DBSEC_DEV_PROF

Profile Creation

```
SQL> create profile dbsec_admin_prof
```

```
2 limit
```

```
3 SESSIONS_PER_USER 5
```

```
4 CONNECT_TIME 4
```

```
5 IDLE_TIME 60
```

```
6 PASSWORD_LIFE_TIME 30
```

```
7 PASSWORD_GRACE_TIME 7
```

```
8 PASSWORD_VERIFY_FUNCTION PASSWORD_COMPLEXITY;
```

OUTPUT:

```
Profile created.
```

```
SQL> create profile dbsec_dev_prof
```

```
2 LIMIT
```

```
3 CONNECT_TIME 720
```

```
4 IDLE_TIME 120
```

```
5 CPU_PER_CALL 1
```

```
6 PASSWORD_LIFE_TIME 30
```

```
7 PASSWORD_GRACE_TIME 14
```

```
8 PASSWORD_VERIFY_FUNCTION PASSWORD_COMPLEXITY;
```

OUTPUT:

```
Profile created.
```

```
SQL> create profile dbsec_clerk_prof
```

```
2 limit
```

```
3 SESSIONS_PER_USER 1
```

```
4 CPU_PER_CALL 60
```

```
5 CONNECT_TIME 480
```

```
6 IDLE_TIME 30
```

```
7 LOGICAL_READS_PER_CALL 1000
```

8 FAILED_LOGIN_ATTEMPTS 3

9 PASSWORD_LIFE_TIME 30

10 PASSWORD_LOCK_TIME 3

11 PASSWORD_GRACE_TIME 14

12 PASSWORD_VERIFY_FUNCTION PASSWORD_COMPLEXITY;

OUTPUT:

```
Profile created.
```

Roles Creation

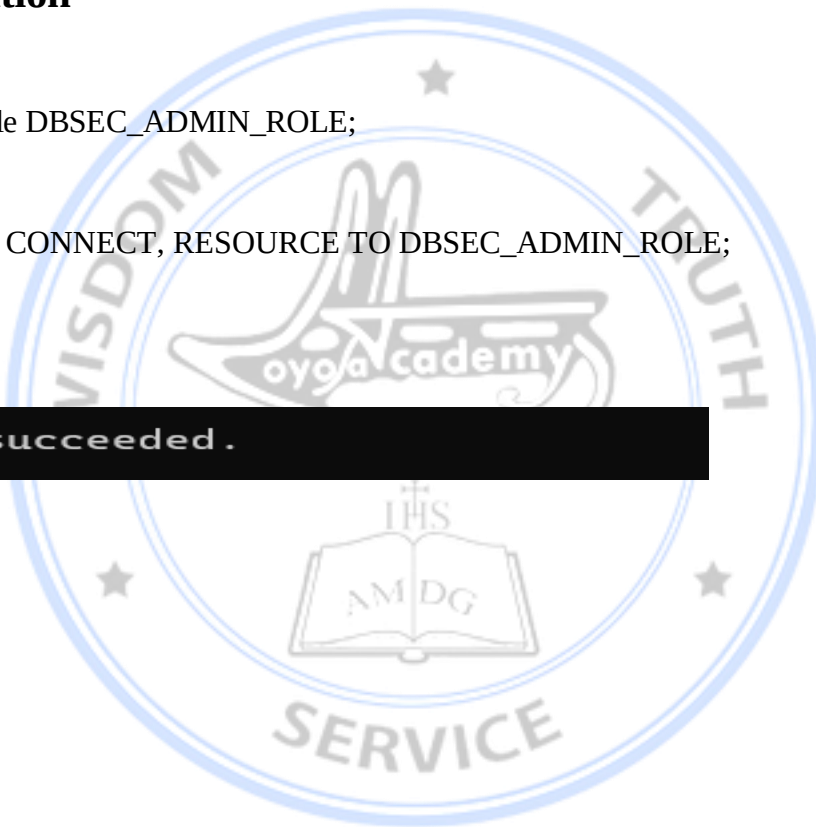
SQL> create role DBSEC_ADMIN_ROLE;

Role created.

SQL> GRANT CONNECT, RESOURCE TO DBSEC_ADMIN_ROLE;

OUTPUT:

```
Grant succeeded.
```



SQL> GRANT CREATE SESSION TO DBSEC_ADMIN_ROLE;

OUTPUT:

```
Grant succeeded.
```

SQL> GRANT SELECT,ALTER ON dbsec.customer to DBSEC_ADMIN_ROLE;

OUTPUT:

```
Grant succeeded.
```

SQL> create role DBSEC_CLERK_ROLE;

Role created.

SQL> GRANT CONNECT, RESOURCE TO DBSEC_CLERK_ROLE;

OUTPUT:

```
Grant succeeded.
```

SQL> GRANT CREATE SESSION TO DBSEC_CLERK_ROLE;

OUTPUT:

```
Grant succeeded.
```

SQL> GRANT SELECT, INSERT,UPDATE ON dbsec.customer to DBSEC_CLERK_ROLE;

OUTPUT:

```
Grant succeeded.
```

SQL> create role DBSEC_SUPERVISOR_ROLE;

Role created.

SQL> GRANT CONNECT, RESOURCE TO DBSEC_SUPERVISOR_ROLE;

OUTPUT:

```
Grant succeeded.
```

SQL> GRANT CREATE SESSION TO DBSEC_SUPERVISOR_ROLE;

OUTPUT:

```
Grant succeeded.
```

SQL> GRANT SELECT, INSERT, UPDATE, DELETE ON dbsec.customer to DBSEC_SUPERVISOR_ROLE;

OUTPUT:

```
Grant succeeded.
```

SQL> create role DBSEC_QUERY_ROLE;

Role created.

SQL> GRANT CONNECT, RESOURCE TO DBSEC_QUERY_ROLE;

OUTPUT:

```
Grant succeeded.
```

SQL> GRANT CREATE SESSION TO DBSEC_QUERY_ROLE;

OUTPUT:

```
Grant succeeded.
```

SQL> GRANT SELECT ON dbsec.customer to DBSEC_QUERY_ROLE;

OUTPUT:

```
Grant succeeded.
```

Assigning Profiles & Roles to Database

Users

Run SQL Command Line

```
SQL> alter user DBSEC_ADMIN  
      profile DBSEC_ADMIN_PROF;
```

OUTPUT:

User altered.

```
SQL> grant DBSEC_ADMIN_ROLE to DBSEC_ADMIN;
```

OUTPUT:



```
Grant succeeded.
```

```
SQL> alter user DBSEC_CLERK  
      profile DBSEC_CLERK_PROF;
```

OUTPUT:

User altered.

```
SQL> grant DBSEC_CLERK_ROLE to DBSEC_CLERK;
```

OUTPUT:



```
Grant succeeded.
```

```
SQL> alter user DBSEC_CLERK  
      profile DBSEC_CLERK_PROF;
```

OUTPUT:

User altered.

SQL> grant DBSEC_CLERK_ROLE to DBSEC_CLERK;

OUTPUT:

```
Grant succeeded.
```

SQL> alter user VPD_CLERK1
profile DBSEC_CLERK_PROF;

OUTPUT:

User altered.

SQL> grant DBSEC_CLERK_ROLE to VPD_CLERK1;

OUTPUT:

```
Grant succeeded.
```

SQL> alter user VPD_CLERK2
profile DBSEC_CLERK_PROF;

OUTPUT:

User altered.

SQL> grant DBSEC_CLERK_ROLE to VPD_CLERK2;

OUTPUT:

```
Grant succeeded.
```

SQL> alter user DBSEC_DEV
profile DBSEC_DEV_PROF;

OUTPUT:

User altered.

```
SQL> grant DBSEC_ADMIN_ROLE, DBSEC_SUPERVISOR_ROLE to DBSEC_DEV;
```

OUTPUT:

```
Grant succeeded.
```

4. Implement view on CUSTOMER table. You will create a VIEW named as CUSTOMER_F_VIEW to display only rows that belong to the log on user. Enable VPD_CLERK1 to access the CUSTOMER data through the view.

Implement virtual private database (VPD) on CUSTOMER table so that only the owner of data can access their own row.

First, you will create a policy function, named "DBSEC_ROW_OWNER" so that only the data that belong to the current user will be accessed. Then, you will add the policy using DBMS_RLS.ADD_POLICY function.

Audit the activities on CUSTOMER table.

Creating view

```
sql>CREATE VIEW CUSTOMER_F_VIEW AS
SELECT * FROM CUSTOMER
WHERE CTL_UPD_USER = USER;
```

OUTPUT:

```
View created.
```

```
SQL>CREATE OR REPLACE FUNCTION DBSEC_ROW_OWNER (
    object_schema IN VARCHAR2,
    object_name IN VARCHAR2
) RETURN VARCHAR2 IS
BEGIN
    RETURN 'CTL_UPD_USER = USER'; -- Restricts access to the logged-in user
END;
/
```

OUTPUT:

```
Function created.
```

Add the policy using DBMS_RLS.ADD_POLICY

SQL>BEGIN

DBMS_RLS.ADD_POLICY

(

object_schema => 'DBSEC',

object_name => 'CUSTOMER',

policy_name => 'CUSTOMER_POLICY',

function_schema => 'DBSEC',

policy_function => 'DBSEC_ROW_OWNER',

statement_types => 'SELECT, INSERT, UPDATE, DELETE',

update_check => TRUE

)

END;

OUTPUT:

pl/sql procedure successful

Grant access to VPD_CLERK

SQL>GRANT SELECT ON CUSTOMER_F_VIEW TO VPD_CLERK;

OUTPUT:

Grant succeeded.

Auditing on customer table

SQL>AUDIT SELECT, INSERT, UPDATE, DELETE ON CUSTOMER BY ACCESS;

OUTPUT:

Audit succeeded.

5. Design and implement the following auditing functions on the CUSTOMER table. Track all "SELECT" activities on the CUSTOMER table, including database user, operating system user, and time when the operation is performed.

Track the changes when CREDIT_LIMIT is set to a value above \$50,000. Track the CUSTOMER table when a customer record was deleted.

Audit the user activities of two users. You will set up an audit mechanism to monitor all activities by two (2) database users: VPD_CLERK1, VPD_CLERK2.

Enable auditing

```
SQL>ALTER SYSTEM SET AUDIT_TRAIL = DB, EXTENDED SCOPE=SPFILE;  
SHUTDOWN IMMEDIATE;  
STARTUP;
```

Audit SELECT on CUSTOMER Table

```
SQL>AUDIT SELECT ON CUSTOMER BY ACCESS;
```

OUTPUT:

```
Audit succeeded.
```

Audit Changes to CREDIT_LIMIT above \$50,000

Create the trigger

```
SQL>CREATE OR REPLACE TRIGGER track_credit_limit_changes  
AFTER UPDATE OF CREDIT_LIMIT ON CUSTOMER  
FOR EACH ROW  
WHEN (NEW.CREDIT_LIMIT > 50000)  
BEGIN  
  
    INSERT INTO audit_credit_limit (user_name, action, table_name, column_name,  
old_value, new_value, change_time)  
  
    VALUES (USER, 'UPDATE', 'CUSTOMER', 'CREDIT_LIMIT', :OLD.CREDIT_LIMIT,  
:NEW.CREDIT_LIMIT, SYSTIMESTAMP);  
  
END;
```

Audit DELETE operations

SQL>AUDIT DELETE ON CUSTOMER BY ACCESS;

OUTPUT:

```
Audit succeeded.
```

Audit activities of users VPD_CLERK1 and VPD_CLERK2

SQL>AUDIT ALL BY VPD_CLERK1, VPD_CLERK2 BY ACCESS;

OUTPUT:

```
Audit succeeded.
```

Check the recorded audit logs:

SQL>SELECT * FROM DBA_AUDIT_TRAIL WHERE OBJ_NAME = 'CUSTOMER';

